Design Amazon

SystemsExpert design walkthrough

Clarifying questions

- Catalog browsing, cart, checkout, inventory, orders?
- Flash sales / hot items?

Functional requirements

- Browse/search products
- Cart management
- Checkout with payment and inventory reservation
- Order history

Non-functional requirements

- Never oversell; never double charge
- Browsing highly available

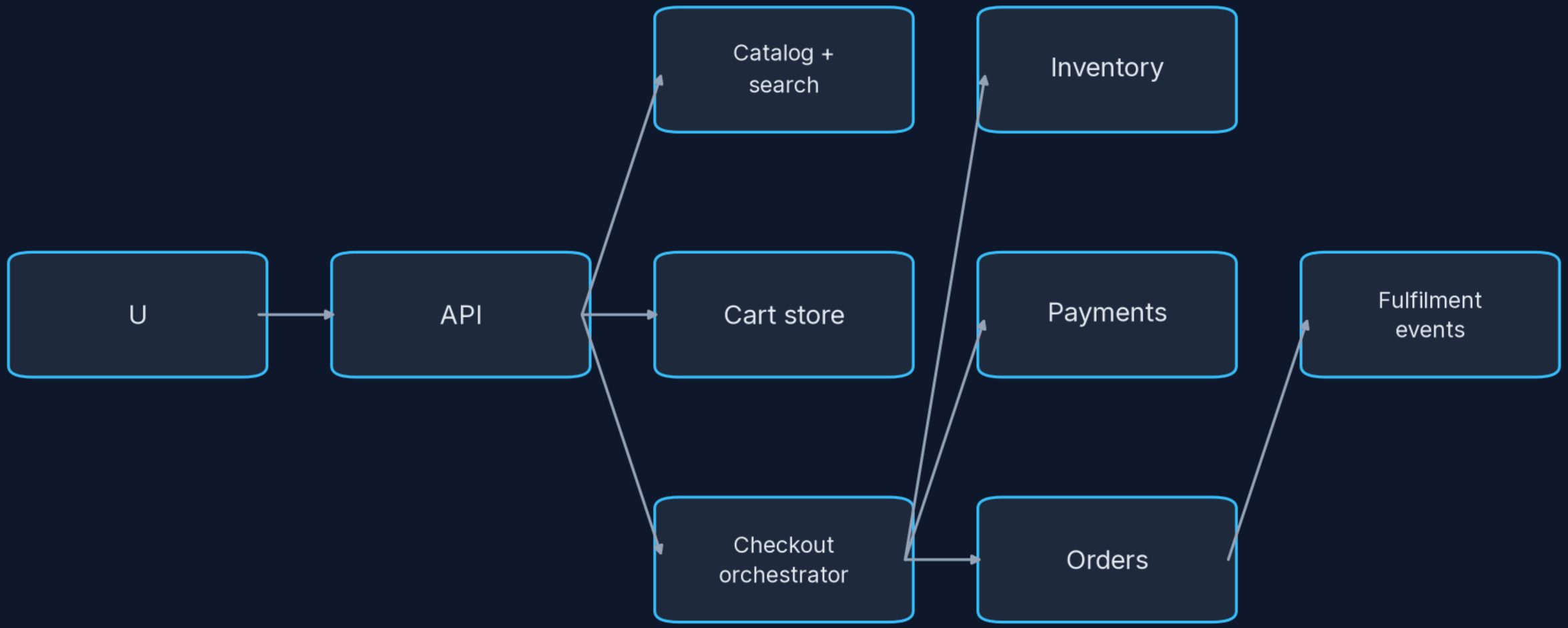
Capacity estimates

- ~100M DAU, ~10M orders/day (~120/s avg, 10x at peaks)

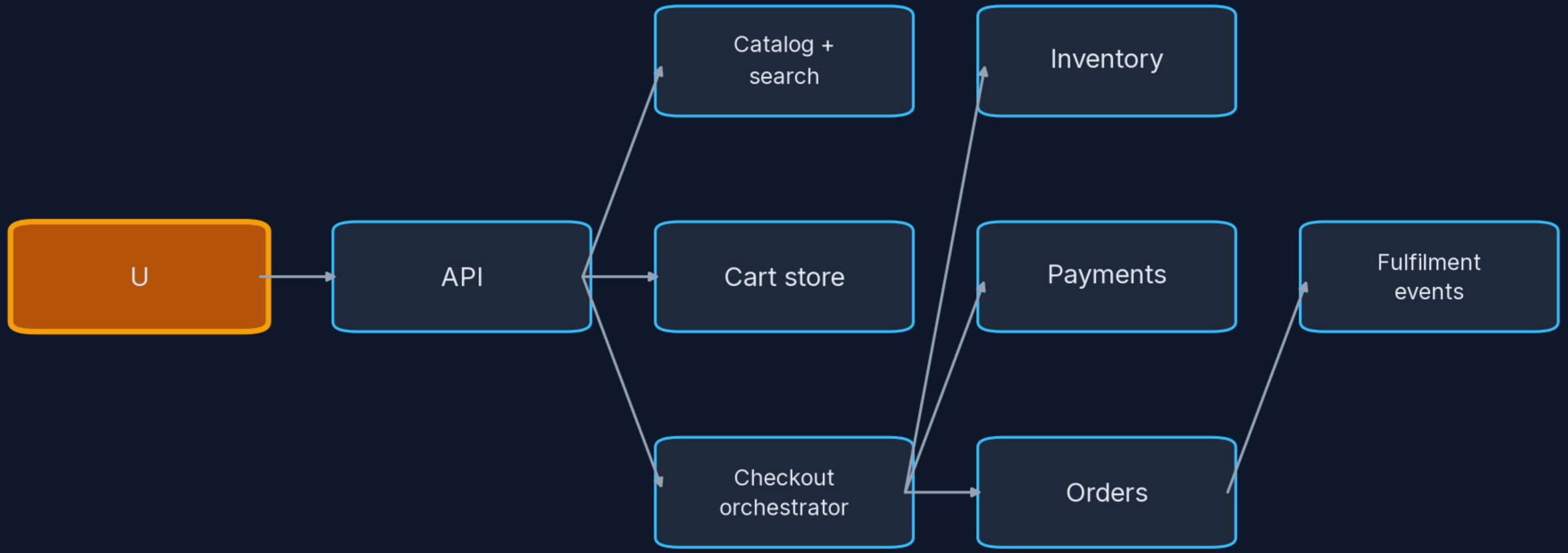
Capacity: worked out

- 10M orders/day ÷ 86,400 ≈ 120 orders/s
- Peak events ~10× ≈ 1.2k orders/s
- 100M DAU × 20 page views ≈ 23k reads/s
- Reads cached; checkout writes must be exact

High-level architecture

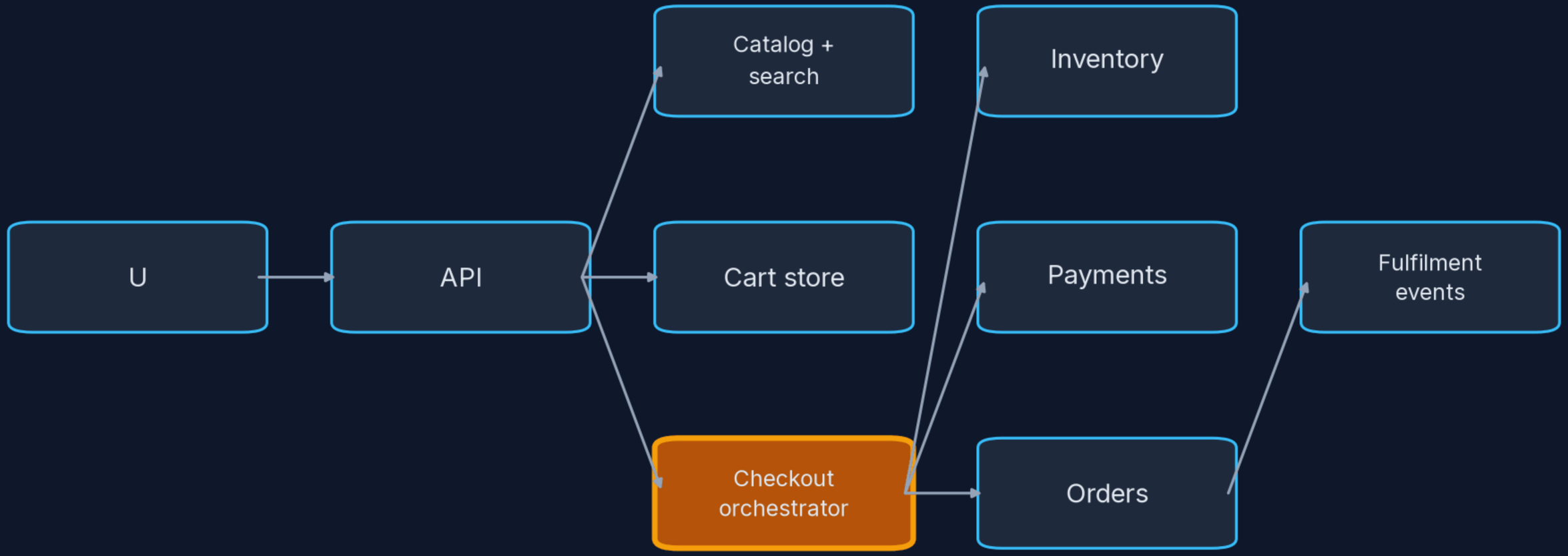


Request flow: Check out



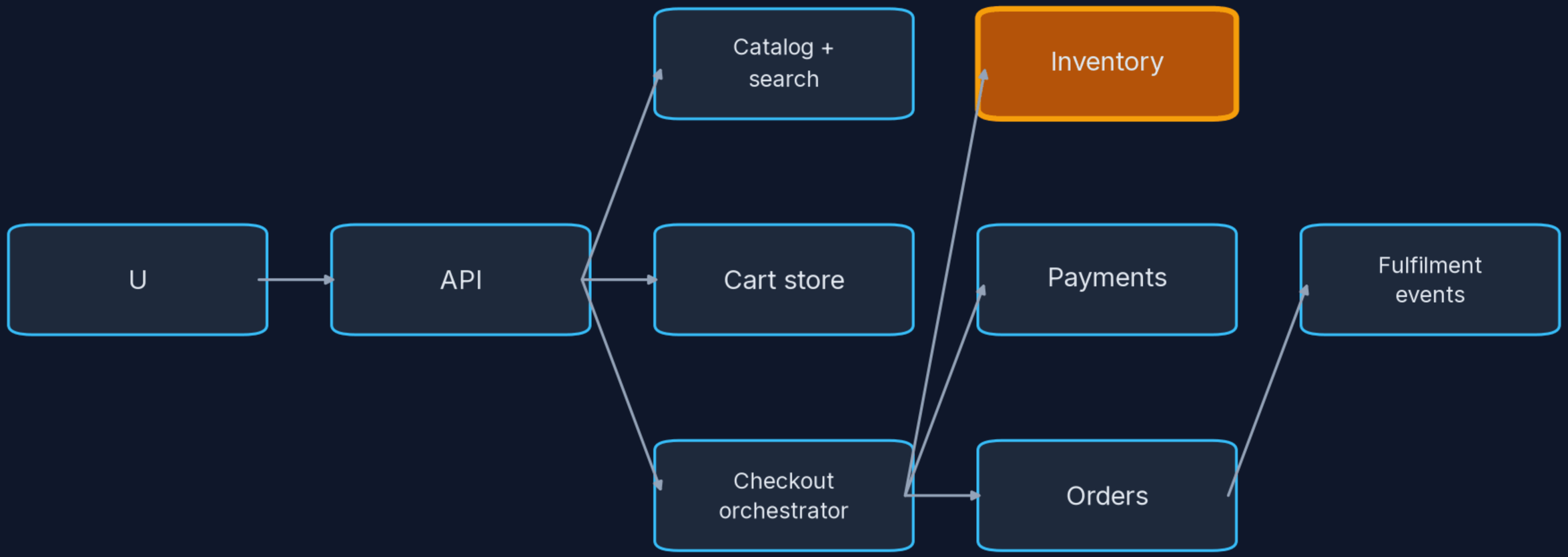
1. User clicks Place order

Request flow: Check out



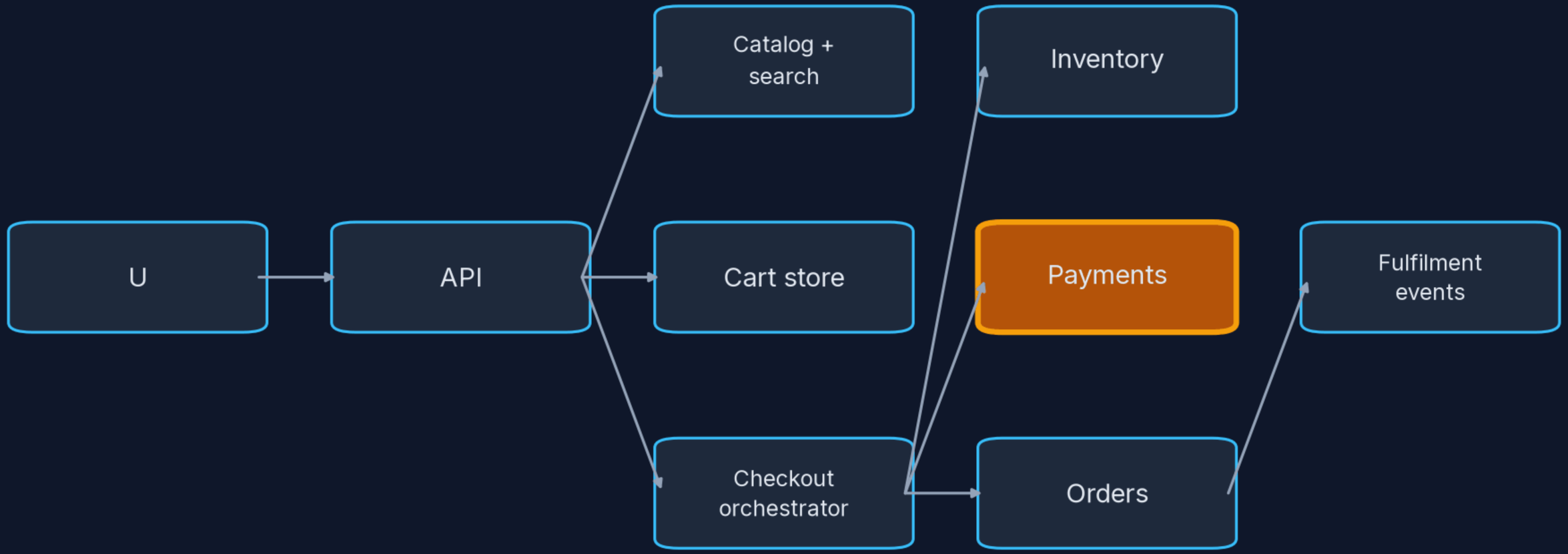
2. Orchestrator starts a saga

Request flow: Check out



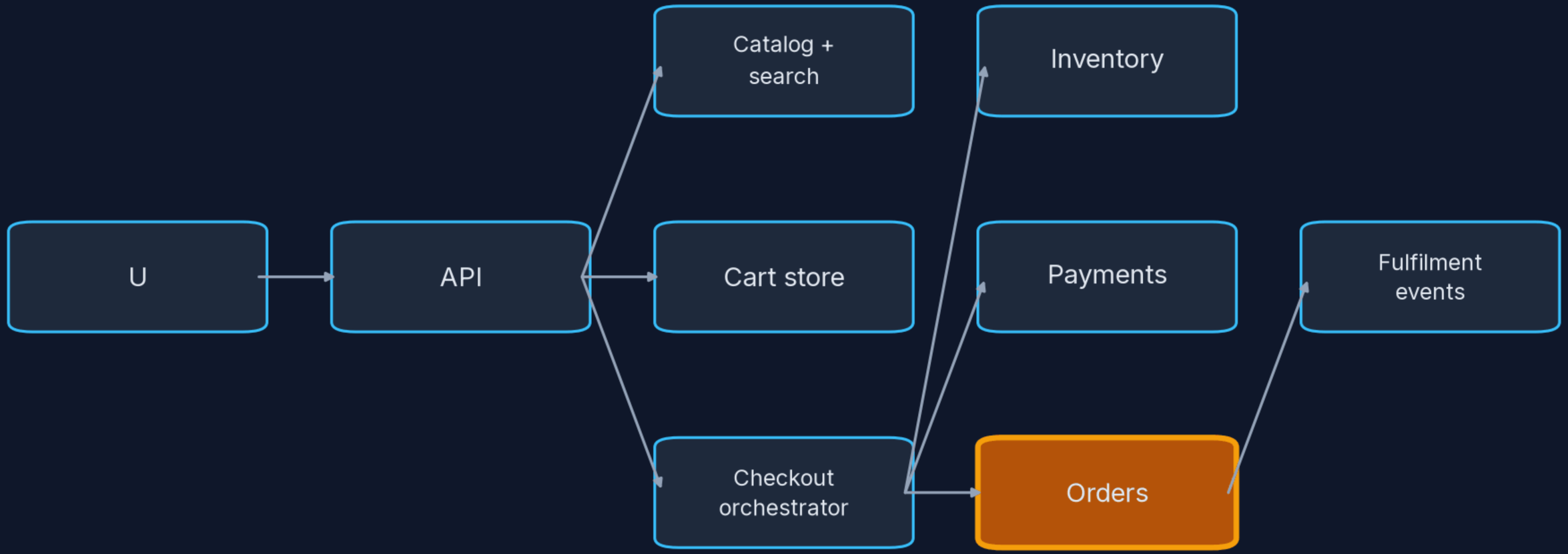
3. Reserve inventory

Request flow: Check out



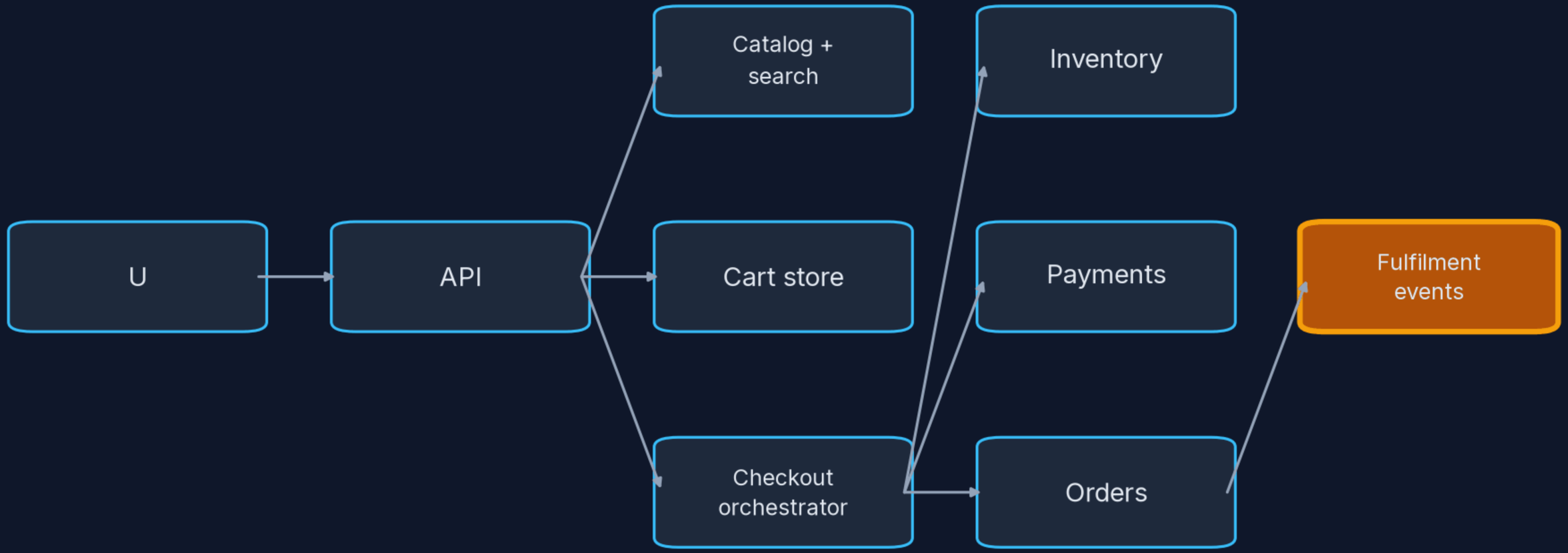
4. Charge payment

Request flow: Check out



5. Create the order

Request flow: Check out



6. Fulfilment begins

Data model

- Inventory(sku, warehouse, available, reserved)
- Orders(id, user_id, items, status, payment_id, idempotency_key)

API

```
POST /cart/items {sku, qty}
POST /checkout {cartId, addressId, paymentMethodId, idempotencyKey} -> order
GET  /orders/{id}
```

Saga

- Reserve inventory → authorise payment → create order → capture; on failure run compensations (release reservation, void auth).

Hot items

- Shard inventory counters; for flash sales use a token queue to admit buyers.

Key decisions

- Saga, not a distributed transaction → services stay independent
- Reserve inventory with expiry → no oversell, no stuck stock
- Idempotent payments → safe retries
- Event-driven fulfilment → decoupled and scalable

Trade-offs & alternatives

- Saga vs 2PC: saga is available and scalable at the cost of temporary inconsistency
- Reservations with TTL avoid inventory lockup

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: U, API, Catalog + search, Cart store, Checkout orchestrator
- Know your trade-offs and failure modes